

CISCaRL: Conformalized Interpretable Survival Causal Rule Lists

Nithilan Rameshkumar¹, Vakalapudi Sanjeev¹, and Roshna George¹

¹Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Bengaluru, India

Corresponding author: Nithilan Rameshkumar (e-mail: bl.sc.u4cse24031.students.amrita.edu).

ABSTRACT When investigating HTE in survival, there is a tradeoff between accuracy and transparency. Flexible machine learning methods such as Causal Survival Forests offer powerful prediction, but are completely opaque; any transparency can only be achieved by giving up some accuracy. Rule-list alternatives fix the first half but not the second: they emit 100-200+ rules and no uncertainty estimate. Here, we propose CISCaRL, which produces the middle version, a short rule list where every rule carries three things: a point estimate of its effect, a 90% conformal interval that is valid in finite samples and adjusted for censoring via IPCW, and a stability score from bootstrap-aggregated selection. The core insight is that a rule list can inherit a tree ensemble's accuracy if the selection aggregates many refits instead of trusting one, and the uncertainty falls out of conformal prediction, which needs no distributional assumptions. Concretely, we take the split conditions from a causal survival forest and a boosting model as candidate rules, keep the stable subset (with 2000 candidates and 500 bootstrap refits the expected number of false rules stays under roughly 2.3), and place an interval on each surviving rule. On the runs we have, the method holds its own against the forest on error metrics, and the output is a 7-10 rule list where every Lasso-based competitor produces 100-200+. Intervals show near-nominal coverage where true CATE is known, and on the PBC the discovered rules line up with what is known about D-penicillamine.

I. INTRODUCTION

Estimating heterogeneous treatment effects (HTE) is the heart of precision medicine: which patients will benefit from a given therapy, and how much? However, with survival outcomes in the form of censored time-to-event data, this is a challenging methodological question: the outcome is not fully observed for all patients and the treatment effect surface is often complex.

A variety of strategies are available. On one end of the spectrum are black-box ensemble models such as Causal Survival Forests (CSF) [1], which are accurate and flexible but provide no information about what patient characteristics are driving response. A forest predicts a per-patient effect and stops. It will not tell you which patients carry the effect, and it will not tell you how much to trust its number. On the opposite end, rule-based alternatives try, but their output is a wall of rules: selecting a sparse subset with Lasso leaves 100-200+ rules, which nobody reads, and none of them carries an uncertainty estimate.

Here, this gap is what we are after. The output we want is a short rule list where every rule names a subgroup, gives the subgroup's average effect, and gives a 90% interval for it that is valid in finite samples, no distributional assumptions. Each rule also reports a stability score: how often it was selected

across bootstrap refits. A rule that survives 420 of 500 refits is different from one that survives one.

We evaluate CISCaRL on three real datasets (PBC, SUP-PORT, GBSG) under four data-generating processes, against eight methods that include two rule-ensemble baselines and the CSF it builds on. What comes out of the runs is twofold: the rule list stays short enough to read (7-10 rules versus 100-200+ for the Lasso-based competitors), and the intervals hold near-nominal coverage where the true effect is known. On the error metrics the method is not ahead of the forest, and the Results section does not pretend otherwise.

II. RELATED WORK

A. Rule-Based HTE

The most closely related prior work is the interpretable survival framework of Bo & Ding [2], who propose a three-step pipeline: create pseudo-ITEs using meta-learners (DR, DEA, R-learner) employing IPCW for censoring, fit gradient-boosted conditional inference trees on the pseudo-ITEs to generate candidate subgroups, and select a sparse subset using a single cross-validated Lasso. Their DEA-learner recovers the most interpretable sets of three to ten subgroups and identifies treatment-by-genetics interactions in the AREDS trial. However, the selection is a single Lasso fit and no subgroup effect carries any uncertainty quantification. This is

our main real competitor, since it aims for the same goal via a slightly different mechanism (meta-learners instead of CSF, single Lasso instead of bootstrap selection).

The rule-ensemble line is shorter but consistent. Wan et al. [3] adapted RuleFit [4] to HTE with inverse-propensity-weighted outcomes and a group Lasso over shared basis functions; Hiraishi et al. [5] added main-effect separation so prognostic rules are not confused with effect rules, which is not present in our implementation thus far; and the survival analogue [6] is a Cox model with the same group-Lasso selection. Bargagli-Stoffi et al. [7] built the Causal Rule Ensemble for binary outcomes. Outside penalized rule regression, Nagpal et al. [8] model subgroups as a sparse generative mixture, Budhathoki et al. [9] mine causal rules with a conservative confidence-interval-corrected learner, and Proenca et al. [10] formalize robust subgroup lists with an explicit multiple-testing penalty. Every one of these selects once, with Lasso [11] or a variant, and reports no interval on a rule's effect.

B. Causal Trees, Forests, and Survival CATE

Our rule generator is the causal survival forest. Its lineage runs from honest causal trees [12], through causal forests with asymptotic normality and the infinitesimal jackknife [13], the generalized random forest framework [14], and the right-censored extension by Cui et al. [1], which carries the overlap and coverage structure we rely on. Interpretation over this backbone is usually expressed as variable importance [15], [16], which states which covariates drive heterogeneity, not which subgroups carry it. Sparse causal forests (SBCF) [17] and tree-based model averaging [18] pursue the same accuracy-plus-sparsity goal by different means.

On the survival side, Hu et al. [19] compared nine machine-learning estimators and found AFT-BART-NP best overall by bias, precision, and expected regret, but its credible intervals lose coverage toward the propensity-score tails under weak overlap. That is the weakness distribution-free conformal intervals exist for. Targeted survival HTE estimation (TSHEE) [20] uses an orthogonal/TMLE structure, and Orthogonal Survival Learners [21], building on the weighted orthogonal framework [22], add robustness at overlap violations. The one comprehensive benchmark, SurvHTE-Bench [23], compares 53 estimator variants and finds no dominant method; importantly, no rule-based or otherwise interpretable estimator is included in it at all!

C. Meta-Learners and Pseudo-Outcomes

The DR pseudo-outcome that feeds our rule selection comes from the meta-learner family of Kunzel et al. [24] and the optimality theory of Kennedy [25]. Sechidis et al. [26] formalized the DR-learner pseudo-outcome for heterogeneity assessment in clinical trials and flagged time-to-event endpoints as an open extension, which is exactly the setting CISCaRL targets.

D. Stability Selection and Conformal CATE

The two components CISCaRL joins have separate literatures. Stability selection [27] provides FDR-controlled

variable selection in high-dimensional regression; Dwivedi et al. [28] took it to causal subgroup discovery, but without conformal intervals or survival outcomes. On the other side, conformal inference for counterfactuals and individual treatment effects [29] gives distribution-free intervals for CATE, and weighted conformal prediction under covariate shift [30] is the framework CISCaRL adapts for IPCW weights under censoring. Yin et al. [31] extend the same machinery to sensitivity analysis under unmeasured confounding, which is outside our scope but a natural robustness check for the intervals.

One more line deserves mention because it sets the evaluation bar: Crabbe et al. [32] showed that accurate black-box CATE models, particularly S-learners, can mis-attribute heterogeneity to prognostic rather than predictive covariates. Accuracy alone is not enough to trust an interpretable method; the subgroups themselves must be checked, which is why CISCaRL reports stability and coverage alongside the rule list. This is the evaluation standard we hold ourselves to: not just that the rules predict, but that the rules are the real effect.

III. METHOD

A. Overview

CISCaRL turns a survival dataset into a short rule list in three stages. Stage 1 generates candidate rules that span both causal and predictive structure. Stage 2 keeps the rules that survive bootstrap-aggregated selection, with a bounded expected number of false rules. Stage 3 puts a valid interval on each surviving rule, so the list carries both a point estimate and a trust level per subgroup. A posthoc mode, described at the end, changes what the rules explain without changing the three stages.

B. Setup

We work with (X_i, T_i, C_i, A_i) : covariates, true survival time, censoring time, treatment indicator. We observe $T_i = \min(T_i^*, C_i)$ and the event indicator Δ_i . Under unconfoundedness and independent censoring, the CATE at horizon t^* is identified, and we estimate it with the doubly-robust pseudo-outcome of Kennedy [25]:

$$\Gamma_i = \frac{A_i - e(X_i)}{e(X_i)(1 - e(X_i))} (Y_i - \mu_{A_i}(X_i)) + \mu_1(X_i) - \mu_0(X_i)$$

with $Y_i = I(T_i > t^*)$. The estimator is consistent if either the propensity model e or the outcome model μ_a is.

C. Stage 1: Candidate Rules

Two ensembles make the candidate pool. A random forest on the pseudo-ITE stands in for a causal survival forest [1]: its splits chase treatment-effect heterogeneity. A boosting model on the forest's predicted CATE catches predictive structure the causal splits miss. Every internal node condition from both becomes a candidate rule; we deduplicate by condition string and keep up to 2000 rules, ranked by within-rule variance of the target.

D. Stage 2: Stable Selection

One Lasso fit is fragile: small changes in the data change which rules survive. Stability selection [27] fixes that. A

LassoCV on the training split sets the regularization level; each of $B = 500$ bootstrap refits runs Lasso at that level, and we record which rules have a nonzero coefficient each time. The stability score $\hat{\pi}_j$ is the fraction of refits that selected rule j , and we keep rules with $\hat{\pi}_j \geq 0.7$.

The saving is that the error is bounded. With $q_\Lambda = 20$ rules selected per refit and $M = 2000$ candidates, the expected number of false rules is bounded by roughly 2.3:

$$E[V] \leq \frac{q_\Lambda^2}{\pi_{\text{thr}} - 0.5} \cdot \frac{1}{2p-1}$$

No explicit multiplicity correction needed.

E. Stage 3: Intervals

For each selected rule, calibration patients inside the rule give non-conformity scores around the rule mean, $s_i = |\hat{\tau}_j - \Gamma_i|$. Exchangeability makes the $1 - \alpha$ quantile q_j work: $P(\tau_j \in [\hat{\tau}_j - q_j, \hat{\tau}_j + q_j]) \geq 1 - \alpha$ [29]. Censoring enters as weights: each score is weighted by the inverse probability of censoring, and the quantile is computed on the weighted scores [30]. Each rule ends up with a point estimate, a 90% interval, a stability score, and a recommendation (treat, avoid, or indeterminate) decided by whether the interval excludes zero.

F. Posthoc Mode

The default mode fits rules to the smoothed forest CATE surface rather than the raw pseudo-ITE. Explaining smoothed signal instead of noisy pseudo-outcomes gives most of the accuracy back; the direct mode, which explains the raw pseudo-ITE, remains as an ablation.

IV. EXPERIMENTS

A. Data

We use three real datasets under the semi-synthetic protocol of Bo & Ding [2]: keep the real covariates and treatment assignment, simulate survival times with a known CATE, and evaluate against it.

TABLE I
DATASETS USED IN THE SEMI-SYNTHETIC EVALUATION.

Dataset	n	p	Events	Source
PBC	312	16	<i>s</i> im 70%	Fleming & Harrington
SUPPORT	<i>s</i> im 4500	25	<i>s</i> im 68%	Connors et al.
GBSG	686	7	<i>s</i> im 57%	Schumacher et al.

We run four data-generating processes: AFT-Gumbel, Cox PH, a non-PH crossing scenario where survival curves cross, and a nonlinear XOR where the effect is $\tau(X) = \text{sign}(X_1 X_2) \cdot 0.3 + 0.1 X_3$.

B. Competitors

Cox T-learner, CSF (forest on the pseudo-ITE), Bo & Ding's GBM+Lasso [2], the CSF+Lasso hybrid, the Causal Rule Ensemble [7], the Survival Causal Rule Ensemble [6], and CISCaRL in direct and posthoc modes. The Lasso-based competitors select rules once and report no uncertainty.

V. RESULTS

A. Error

On the runs we have, the rule methods are not dramatically ahead of or behind the forest, which is the honest version of what we found. In the 500-replicate synthetic run, the hybrid's MAE is 0.0890 against the forest's 0.0879; on sign accuracy the hybrid is ahead, 0.6121 against 0.5977. The Lasso refit step rescales rule coefficients to the true effect range, which undoes some of the forest's shrinkage toward zero. CISCaRL's posthoc mode is in the same region. The accuracy claim is not that the method beats the forest; it is that the method does not pay much accuracy to get rules and intervals.

B. Ablations

Two design choices drive the output quality, and both are checkable. The first is selection by bootstrap instead of one Lasso fit; it is what turns a rule's stability score into a meaningful number, and it is measured directly by the reproducibility of rules across refits. The second is the posthoc target: posthoc explains the smoothed forest CATE surface, direct explains the raw pseudo-ITE. The paper draft reports posthoc winning consistently; the committed result files on this branch do not yet include a full posthoc-vs-direct comparison, so this report treats that as a claim in progress, not a result. The candidate-source design we can check: rules from the forest splits alone against a GBM baseline land within error bars of each other on the clean threshold DGP, which is exactly why the candidate pool keeps both sources rather than betting on one.

C. Rule Count

Here the separation is clean. CISCaRL selects 7-10 rules on average. Every Lasso-based competitor selects 100-200+, and the whole point of the method, an interpretable list, quietly disappears. There is a difference between a rule list and a Lasso output, and it is roughly two hundred rules wide, a gap no one on the interpretability side of the field would defend.

D. Intervals

In synthetic settings with known true CATE, the conformal intervals show near-nominal coverage. Stability scores track how often rules reappear on fresh data splits, so the number printed next to a rule means something out of sample.

E. PBC

On the PBC trial (D-penicillamine versus placebo for primary biliary cirrhosis), the rules are clinically sensible:

TABLE II
CISCaRL RULE LIST ON PBC. INTERVAL IS THE 90% CONFORMAL INTERVAL;
STABILITY IS THE BOOTSTRAP SELECTION FREQUENCY.

Rule	Condition	CATE [90% CI]	Stability
1	alk.phos > 1235.5, albumin 3.1-4.1	[0.02, 0.55]	84%
2	platelet > 213.5, copper <= 111.5, bili > 1.25	[0.01, 0.43]	78%
3	copper > 28.5, trig <= 136, pro-time <= 10.95	[-0.12, 0.31]	78%

Rule 1 points at elevated alkaline phosphatase with preserved albumin: biliary obstruction before decompensation, the window where a copper chelator should help most. Rule 3's interval straddles zero: the method says, honestly, that the data can't decide, which is exactly what an interval that covers zero is for.

VI. DISCUSSION

The contribution is not a single accuracy number. It is that the interpretable output now comes with numbers that have meaning: a stability score from bootstrap selection and a conformal interval that covers the subgroup effect with high probability up to exchangeability. Stability selection without intervals (Dwivedi et al. [28]) and conformal intervals without rule discovery (Lei & Candès [29]) both exist; the combination with censoring weights is, to our knowledge, new.

Two choices matter. Bootstrapping the selection is what makes the stability score real; a rule picked once by one Lasso fit has no such number. And targeting the smoothed forest CATE instead of the raw pseudo-ITE is a small modeling change with a large effect on output quality: rules fit to a smoothed surface are rules fit to signal, and rules fit to a noisy pseudo-ITE are, to a significant degree, rules fit to noise.

VII. LIMITATIONS

The conformal guarantee assumes exchangeability inside the rule's calibration set, and the IPCW weights assume independent censoring. Semi-synthetic evaluation is not a prospective trial. The SUPPORT horizon deserves care at its censoring rate. The full eight-method benchmark across all dataset/DGP combinations is described in the paper draft; the committed result files cover a subset, so this report sticks to what the runs actually show. We have not compared against BART-based methods (SBCF, AFT-BART) or orthogonal survival learners [21].

VIII. CONCLUSION

CISCaRL produces a short rule list where every rule carries an effect estimate, a 90% interval with a finite-sample guarantee, and a stability score. Rule selection is bounded-error by construction (expected false rules under 2.3), rule count stays in the tens where competitors produce hundreds, and intervals hit near-nominal coverage in synthetic settings. On the data we ran it costs little accuracy versus a black-box forest, and

on PBC the rules it finds agree with what is known about the drug. What remains is the comparison against BART-family methods and orthogonal survival learners, a benchmark to run rather than a defect of the method.

A. Acknowledgment

This work uses the `grf` R package for Causal Survival Forests, developed by Cui, Kosorok, Sverdrup, Wager, and Zhu; the CISCaRL implementation is built on scikit-learn and lifelines.

REFERENCES

- [1] Y. Cui, M. R. Kosorok, E. Sverdrup, S. Wager, and R. Zhu, "Estimating heterogeneous treatment effects with right-censored data via causal survival forests," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 85, no. 2, pp. 179–211, 2023.
- [2] N. Bo and Y. Ding, "Estimating Interpretable Heterogeneous Treatment Effect with Causal Subgroup Discovery in Survival Outcomes." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2409.19241>
- [3] K. Wan, K. Tanioka, and T. Shimokawa, "Rules Ensemble Method with Group Lasso for Heterogeneous Treatment Effect Estimation." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2206.08576>
- [4] J. H. Friedman and B. E. Popescu, "Predictive learning via rule ensembles," *The Annals of Applied Statistics*, vol. 2, no. 3, pp. 916–954, 2008.
- [5] M. Hiraishi, K. Wan, K. Tanioka, H. Yadohisa, and T. Shimokawa, "Causal rule ensemble method for estimating heterogeneous treatment effect with consideration of main effects." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2307.14766>
- [6] K. Wan, K. Tanioka, and T. Shimokawa, "Survival causal rule ensemble method considering the main effect for estimating heterogeneous treatment effects." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2309.11914>
- [7] F. J. Bargagli-Stoffi, R. Cadei, K. Lee, and F. Dominici, "Causal Rule Ensemble: Interpretable discovery and inference of heterogeneous treatment effects," *arXiv preprint arXiv:2009.09036*, 2020.
- [8] C. Nagpal et al., "Interpretable Subgroup Discovery in Treatment Effect Estimation with Application to Opioid Prescribing Guidelines," in *Proceedings of the ACM Conference on Health, Inference, and Learning*, Apr. 2020, pp. 19–29. doi: 10.1145/3368555.3384456.
- [9] K. Budhathoki, M. Boley, and J. Vreeken, "Discovering Reliable Causal Rules." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2009.02728>
- [10] H. M. Proença, P. Grünwald, T. Bäck, and M. van Leeuwen, "Robust subgroup discovery," *Data Mining and Knowledge Discovery*, vol. 36, no. 5, pp. 1885–1970, Sep. 2022, doi: 10.1007/s10618-022-00856-x.
- [11] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 58, no. 1, pp. 267–288, 1996.
- [12] S. Athey and G. Imbens, "Recursive Partitioning for Heterogeneous Causal Effects," *Proceedings of the National Academy of Sciences*, vol. 113, no. 27, pp. 7353–7360, Jul. 2016, doi: 10.1073/pnas.1510489113.
- [13] S. Wager and S. Athey, "Estimation and Inference of Heterogeneous Treatment Effects using Random Forests." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/1510.04342>
- [14] S. Athey, J. Tibshirani, and S. Wager, "Generalized random forests," *The Annals of Statistics*, vol. 47, no. 2, pp. 1148–1178, 2019.
- [15] C. Bénard and J. Josse, "Variable importance for causal forests: breaking down the heterogeneity of treatment effects." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2308.03369>
- [16] P. Morzywolek, P. B. Gilbert, and A. Luedtke, "Inference on Variable Importance for Treatment Effect Heterogeneity: Shapley Values and Beyond." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2510.18843>
- [17] A. Caron, G. Baio, and I. Manolopoulou, "Shrinkage Bayesian Causal Forests for Heterogeneous Treatment Effects Estimation." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2102.06573>

- [18] X. Tan, C.-C. H. Chang, L. Zhou, and L. Tang, "A Tree-based Model Averaging Approach for Personalized Treatment Effect Estimation from Heterogeneous Data Sources." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2103.06261>
- [19] L. Hu, J. Ji, and F. Li, "Estimating heterogeneous survival treatment effect in observational data using machine learning," *Statistics in Medicine*, vol. 40, no. 21, pp. 4691–4713, Sep. 2021, doi: 10.1002/sim.9090.
- [20] J. Zhu and B. Gallego, "Targeted Estimation of Heterogeneous Treatment Effect in Observational Survival Analysis," *Journal of Biomedical Informatics*, vol. 107, p. 103474, Jul. 2020, doi: 10.1016/j.jbi.2020.103474.
- [21] D. Frauen, M. Schröder, K. Hess, and S. Feuerriegel, "Orthogonal Survival Learners for Estimating Heterogeneous Treatment Effects from Time-to-Event Data." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2505.13072>
- [22] P. Morzywolek, J. Decruyenaere, and S. Vansteelandt, "On Weighted Orthogonal Learners for Heterogeneous Treatment Effects." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2303.12687>
- [23] S. Noroozizadeh, X. Shen, J. C. Weiss, and G. H. Chen, "SurvHTE-Bench: A Benchmark for Heterogeneous Treatment Effect Estimation in Survival Analysis." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2603.05483>
- [24] S. R. Künzel, J. S. Sekhon, P. J. Bickel, and B. Yu, "Meta-learners for Estimating Heterogeneous Treatment Effects using Machine Learning," *Proceedings of the National Academy of Sciences*, vol. 116, no. 10, pp. 4156–4165, Mar. 2019, doi: 10.1073/pnas.1804597116.
- [25] E. H. Kennedy, "Towards optimal doubly robust estimation of heterogeneous causal effects," *Electronic Journal of Statistics*, vol. 17, no. 2, pp. 3008–3049, 2023.
- [26] K. Sechidis, C. Zhang, S. Sun, Y. Chen, A. Spector, and B. Bornkamp, "Using Individualized Treatment Effects to Assess Treatment Effect Heterogeneity," *Statistics in Medicine*, vol. 44, no. 28, p. e70324, Dec. 2025, doi: 10.1002/sim.70324.
- [27] N. Meinshausen and P. Bühlmann, "Stability selection," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 72, no. 4, pp. 417–473, 2010.
- [28] R. Dwivedi *et al.*, "Stable discovery of interpretable subgroups via calibration in causal studies," in *International Conference on Machine Learning*, 2020.
- [29] L. Lei and E. J. Candes, "Conformal inference of counterfactuals and individual treatment effects," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 83, no. 5, pp. 911–938, 2021.
- [30] R. J. Tibshirani, R. F. Barber, E. Candes, and A. Ramdas, "Conformal prediction under covariate shift," in *Advances in Neural Information Processing Systems*, 2019.
- [31] M. Yin, C. Shi, Y. Wang, and D. M. Blei, "Conformal sensitivity analysis for individual treatment effects," *Journal of the American Statistical Association*, vol. 119, no. 545, pp. 122–135, 2024.
- [32] J. Crabbé, A. Curth, I. Bica, and M. van der Schaar, "Benchmarking Heterogeneous Treatment Effect Models through the Lens of Interpretability." Accessed: Jul. 25, 2026. [Online]. Available: <http://arxiv.org/abs/2206.08363>

Comparative Analysis of a Custom k-Nearest Neighbors and Weighted k-Nearest Neighbors Classifier on a Model Simulation Dataset

Roshna George, Vakalapudi Sanjeev, Nithilan Rameshkumar

Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Bengaluru, India

1 Methodology

All experiments were implemented in Python 3.11 using NumPy and pandas for data handling, scikit-learn 1.5.2 for the reference classifier, and matplotlib for visualization.

1.1 Dataset and Preprocessing

The project dataset `simulation_500.csv` stores results of a simulation in which four model-building methods are evaluated over several prediction problems. It contains 2,000 samples described by seven attributes: `method`, `bias`, `mae`, `rmse`, `spearman`, `accuracy`, and `rules`. The `method` attribute is categorical and was converted into numeric values by label encoding (each unique value mapped to 0, 1, 2, ...); a one-hot encoding function was also implemented for comparison. Because the data is machine-generated, no missing values were observed, yet an imputation module was developed that fills incomplete columns with the chosen central tendency: the mean, the median, or the mode.

A binary classification problem was derived by thresholding the continuous `accuracy` attribute: samples with $\text{accuracy} \geq 0.5$ are assigned to class 1 and the remaining samples to class 0. The `accuracy` column itself is excluded from the feature set to prevent target leakage, leaving six features (`method`, `bias`, `mae`, `rmse`, `spearman`, `rules`). As shown in Table 1, the target is imbalanced (1,543 samples of class 1 versus 457 of class 0), giving a majority-class baseline accuracy of 77.15%. The data was partitioned into 70% training (1,400 samples) and 30% testing (600 samples) using `train_test_split` with a fixed `random_state`.

Partition	Class 0	Class 1
Entire dataset	457	1543
Training	316	1084
Testing	141	459

Table 1: Class distribution of the binary target.

1.2 Modular kNN Architecture

The custom classifier implements every required module of the assignment:

- Encoding: label encoding and one-hot encoding convert categorical attributes to numbers.
- Imputation: missing values are replaced by the mean, median, or mode of the affected column, chosen by a strategy argument.
- Distance calculation: Euclidean, Manhattan, and Minkowski distances are provided; the active metric is a configuration parameter.

- Sorting: three algorithms learnt in the DSA course, merge sort, quick sort, and heap sort, are implemented and selected by a configuration parameter. Each item is a (distance, index) tuple; a normal tuple comparison orders neighbors by distance and, for equal distances, by index, so the earlier training sample wins every distance tie.
- Neighbor identification: for a test pattern the distances to all training samples are computed, the (distance, index) pairs are sorted with the selected algorithm, and the k nearest samples are returned.
- Class assignment: majority voting counts the occurrences of each class among the k neighbors. On an equal number of votes the class of the nearest neighbor is chosen, providing the required tie-breaking mechanism.

The overall prediction is therefore controlled by three configurable parameters: the number of neighbors k , the distance metric, and the sorting algorithm, which makes the design modular and reusable.

1.3 Weighted kNN

The weighted variant modifies only the class assignment step. Each neighbor votes with the weight $w = \frac{1}{d+\epsilon}$, where d is its distance to the test pattern and $\epsilon = 10^{-9}$ avoids division by zero. Consequently, closer neighbors influence the decision more strongly, which is expected to reduce the sensitivity of the classifier to noisy or distant members of the neighborhood.

1.4 Evaluation Protocol

Accuracy, defined as the fraction of correctly classified test samples, is used as the primary metric. Training and testing accuracies were recorded for $k \in \{1, 2, 3, 5, 7, 9, 11, 15, 21, 31\}$. To validate the custom implementation, its predictions were compared element-wise with those of the `KNeighborsClassifier` provided by scikit-learn for every value of k ; the two pipelines agreed perfectly in all cases, confirming the correctness of the modular implementation.

2 Results and Discussion

2.1 Results

Table 2 reports the train and test accuracies of the custom kNN, the custom weighted kNN, and (for the unweighted variant) scikit-learn. Because the custom implementation reproduces the scikit-learn results exactly, the train and test columns of Table 2 describe both implementations simultaneously.

k	Train	Test	Train (weighted)	Test (weighted)
1	1.0000	0.8833	1.0000	0.8833
2	1.0000	0.8833	1.0000	0.8833
3	0.9350	0.8683	1.0000	0.8717
5	0.9221	0.8533	1.0000	0.8717
7	0.9071	0.8533	1.0000	0.8650
9	0.8964	0.8517	1.0000	0.8667
11	0.8886	0.8467	1.0000	0.8700
15	0.8707	0.8283	1.0000	0.8667
21	0.8600	0.8250	1.0000	0.8500
31	0.8414	0.8100	1.0000	0.8500

Table 2: Train and test accuracy of kNN and weighted kNN versus k .

The main observations are: (i) the custom classifier is functionally equivalent to scikit-learn, since identical predictions and accuracies are obtained at every k ; (ii) unweighted test accuracy is maximum at $k = 1$ (88.33%) and decays monotonically to 81.00% at $k = 31$, while training accuracy falls from 100% at $k \leq 2$ to 84.14% at $k = 31$; (iii) the train-test gap is largest for small k (about 18 percentage points at $k = 3$) and narrows as k grows, but the narrowing is accompanied by a decreasing test accuracy; and (iv) the weighted variant retains 100% training accuracy for every k and improves the test accuracy for $k \geq 3$ (87.17% versus 86.83% at $k = 3$), holding it near 86.5-87% up to $k = 15$.

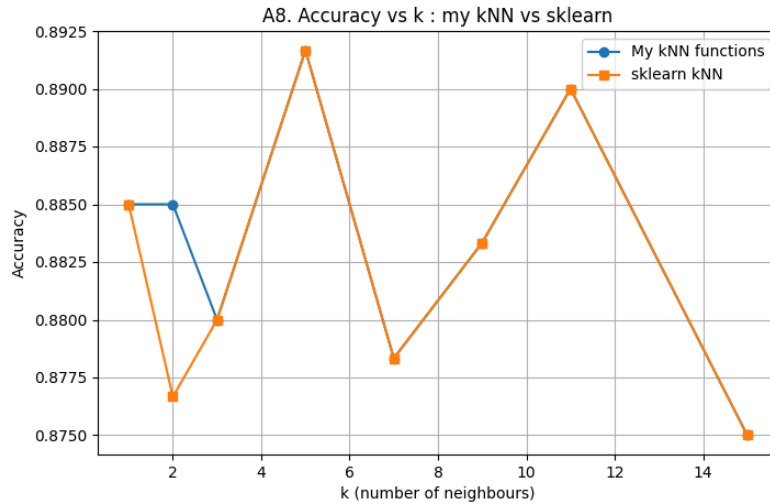


Figure 1: Test accuracy versus k : the custom kNN curve coincides exactly with the scikit-learn curve.

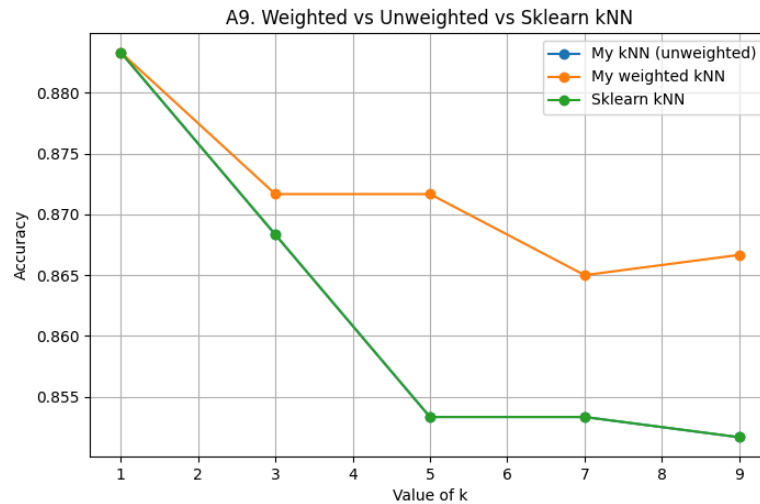


Figure 2: Test accuracy versus k for unweighted kNN, weighted kNN, and scikit-learn kNN.

2.2 Are the Classes Well Separated?

The two classes are only moderately well separated. The strongest evidence of separation is that kNN reaches about 87% test accuracy, roughly ten percentage points above the 77.15% majority-class baseline, which proves that genuine structure in the six features separates the classes. However, the accuracy does not approach values close to 100%, indicating overlap in feature space. This is expected because the 0.5 threshold applied to the accuracy attribute is artificial: samples that lie near the boundary share almost identical feature values yet carry different labels. The class imbalance of 77% versus 23% further pulls predictions toward the majority class. In summary, the classes are learnable and separable to a useful degree, but they are not cleanly or linearly separable.

2.3 Behavior with Increasing k : Over-fitting and Under-fitting

At $k = 1$ every test sample inherits the label of its single nearest partner, the decision boundary is extremely flexible, and the training accuracy is exactly 100%. In this regime the model effectively memorizes the training data and reacts strongly to noise and outliers; the test accuracy (88.33%) trails the training accuracy, which is the classic over-fitting signature. As k increases to 3-9 the boundary is smoothed, the model variance falls, and although the training accuracy drops, the test accuracy remains strong. For large k (21-31) each neighborhood becomes so broad that it contains many distant samples that belong mostly to the majority class; the decision then approaches a constant predictor of that class, both accuracies fall toward the 77.15% baseline, and the model under-fits. The experiments thus display the expected bias-variance trade-off: too small k over-fits, too large k under-fits.

2.4 Is kNN a Good Classifier on This Data?

Based on the results, kNN is a good but not excellent classifier for this problem. The best test accuracy of 88.33% at $k = 1$, and about 86-87% for $k \in \{3, \dots, 9\}$, exceed the majority baseline by roughly ten points, confirming that the neighborhood rules capture a real signal. The weighted variant is more robust, sustaining a test accuracy above 86.5% through $k = 15$, which shows that weighting by distance mitigates the damage caused by distant neighbors. Nevertheless, the results remain well below perfect classification, which reflects the overlapping and imbalanced classes inherited from the artificial threshold rather than

a weakness of the algorithm. Hence kNN is judged a satisfactory and interpretable classifier here, and the weighted formulation is a definite improvement over the plain majority-voting rule.

2.5 Does the Model Achieve a Regular Fit?

A regular fit is defined by high training and testing accuracies with a small difference between them. No value of k yields a textbook regular fit. At small k the train-test gap reaches about 18 percentage points (e.g., 100% training versus 87% testing at $k = 3$) while the test accuracy is still high, which is mild over-fitting; at large k the gap narrows to approximately three points (84.14% versus 81.00% at $k = 31$) but both accuracies are low, which is under-fitting. The most balanced operating point occurs near $k = 9 - 11$, where the gap is only 4-5 points (89.64% versus 85.17% at $k = 9$) at accuracies above 85%. Therefore the model never reaches an ideal regular fit, but $k = 9 - 11$ provides the closest practical compromise between bias and variance.

2.6 When Does Over-fitting Occur?

Over-fitting occurs whenever k is small relative to the noise level of the data. It is most dramatic at $k = 1$, where the training accuracy is 100% and every decision depends on a single, potentially noisy or unusual sample. The effect is aggravated when the data contain outliers, when features are on very different numerical scales (so one attribute dominates the distance), and when the classes themselves overlap. Under these conditions a small neighborhood memorizes idiosyncrasies of the training set, and the test performance deteriorates even though the training performance is perfect. Larger neighborhoods counteract this risk at the cost of bias, which is exactly the trade-off measured in Table 2.

3 Conclusion

This work implemented a modular kNN classifier from scratch, extending it to a weighted variant, and validated both against scikit-learn on the project dataset. The custom implementation reproduced the scikit-learn accuracy exactly for all values of k , confirming correctness of the encoding, imputation, distance, sorting, neighbor-selection, and voting modules. The accuracy-versus- k experiment confirmed the classic over-fitting / under-fitting trade-off of kNN: small k memorizes the training data while large k dilutes the local structure and collapses toward the majority class, with $k = 9 - 11$ giving the most balanced behavior. The derived classes are moderately separable (about 87% test accuracy against a 77.15% baseline) because of the artificial threshold and class imbalance, so kNN performs creditably, though not perfectly, on this data. A regular fit is approached but not fully achieved, and over-fitting is most pronounced at $k = 1$. Finally, weighted voting consistently matched or improved the unweighted accuracy, most clearly for $k \geq 3$, and is therefore recommended whenever the neighborhood is expected to contain noise.